INTRODUCTION TO VISUAL AND INTERACTIVE PROGRAMMING

GROUP ASSIGNMENT PROJECT

INTAKE	:	UCDF2308ICT, UCDF2308ICT(SE), UCDF2308ICT(DI), UCDF2308ICT(ITR)
LECTURER	:	Ms. AZIAH BINTI ABDOLLAH
DUE DATE (GROUP PROJECT)	:	Week 14 (Application and Documentation)

CLO2	Apply the elements of computational thinking to solve a problem (C3, PLO2)	Project-Documentation
CLO3	Use the visual interactive programming tools to develop an application. (A1, PLO6)	Project - Implementation

1.0 ASSESSMENT DETAILS:

GROUP PROJECT (60%)

Work with your group to create a visual and interactive application using SNAP! based on the following guidelines:

1. You are required to create an interactive visual application based on the case study below.
2. You are advised to use your own logic flow to create the application.
3. Interactive **game** applications are **NOT** allowed.
4. You are required to form a team (in a group of **THREE (3)** to **FOUR (4)** persons)
5. You are required to choose **ONE (1)** case study either **CASE STUDY A** or **CASE STUDY B** and agreed by all group members.

CASE STUDY A: STUDENT GRADING SYSTEM

Program Specification:

Academic performance of student was assessed and graded at the end of each semester with the prescribed grading system by the university. You are required to develop one Student Grading System to calculate the overall marks of the student's assessments.

There are 3 end users for this system, 1. Admin 2. Lecturer 3. Students. Your system should have the following characteristics. Each of the characteristics will be divided based on what you have learnt during the class as follow:

Part 1: Display Menu

1. Create a main menu for your grading system. The main menu will display the system's main screen.

Part 2: *Add, View* and *Validate* Student's Record

1. Your system should first validate the chosen menu by user and if the chosen menu matches as displayed in Part 1, the relevant menu will be open.
2. Users need to enter the student's name, student ID, course name, assignment, mid-test, and final exam. The total marks calculated based on assignment is 30%, mid-test is 20% and final exam is 50%.
3. Your system should first check if the value entered for the marks are valid or not (no text value is allowed). If invalid, the program should ask the user to re-enter the valid value.
4. If the assignment score is not between 0 and 30, or the mid-test score is not between 0 and 20, or exam score is not between 0 and 50, the program should display an error message and ask the user to re-enter the scores.
5. For the student to pass the module, they must obtain an overall score of at least 50 marks.
6. The grade must be calculated as per following rules:

Total Marks	Grade
80 – 100	A+
75 – 79	A
70 – 74	B+
65 – 69	B
60 – 64	C+
55 – 59	C
50 – 54	C-
40 – 49	D
30 – 39	F+
20 – 29	F
0 – 19	F-

7. Display the student records.

Part 3: List and Search Record

1. Your system should add all the new student records into the list.

2. Users can search specific student records. Users also can search for the highest and lowest grade.
3. Users can view all student records from the list.

CASE STUDY B: HOTEL RESERVATION SYSTEM

Program Specification:

You are required to develop a Hotel Reservation System to handle room booking for the customer. The system could perform the hotel booking, provide customers with hotel rooms info, room service, billing and record keeping.

There are 2 end users for this system, 1. Admin 2. Customers. Your system should have the following characteristics. Each of the characteristics will be divided based on what you have learnt during the class as follow:

Part 1: Display Menu

Create a main menu for your hotel reservation system. The main menu will display the system's main screen.

Part 2: *Add, View,* and *Validate* the room booking

1. Your system should first validate the chosen menu by user and if the chosen menu matches as in Part 1, the relevant menu will be open.
2. Users need to enter the booking room details. Your system should provide customers with hotel room information.
3. Users can add room service to order the food from the hotel's restaurant. Room service menu will provide the customer with the restaurant's menu card.
4. Users can generate the hotel room and restaurant bill upon checkout.
5. Your system should first check if the value entered for the number are valid or not (no text value is allowed). If invalid, the program should ask the user to re-enter the valid value.

Part 3: List and Search Record

1. Your system should add all the booking details into a list
2. Users can search their booking history
3. Users can view the room service menu.
4. Users can view all the room details.

DO NOT PANIC: You do not have to be an artist or a designer to complete the project.

Do's	Don'ts
Make it humorous	No blood & guts, no spewing
Add details & make your application more expressive	No violence, no loping heads, no missing limbs
Use ALL skills learned	No jumpy application, unless it's intentional and a small part of the overall application
Different scene as background images that develop your user interface	No bathroom humor
	No stick figures. (Try to be more creative in the use of ALL tools)
	Don't just use old stick figure skills.

Select **ONE (1)** person from your group to be the group leader.

As a Group Leader, you are required to:

- Monitor and manage the works and tasks among your group members.
- Ensure the whole application is functioning along with the full completion of the documentation.
- Compile all your group tasks and works into a complete set of documentation.
- Report to the lecturer if any group member is not being cooperative in the group.

PART A – APPLICATION (45%)

Your application must include each of the following:

Interactive. The application must be interactive. Users must be able to interact with your application. Avoid making a lot of animation, where the user does not control the program.

Technical Requirements:

- **Sprites –** Your application must have multiple characters(sprites), **not including** any sprites used to give instructions. **Do not use** *Alonzo (BYOB), Cat (Scratch)* or *Turtle (Snap)* sprite. You need to create or edit at least **ONE (1)** sprite.
- **Stage / Scene** – Have more than **TWO (2)** a different background.
- **Repetition** – Your application must have at least **TWO (2)** loops with a variable other than **"forever"** somewhere in the project.
- **Decision Making** - Your application must have at least **TWO (2) "if"** or **"if-else"** blocks. It is **compulsory** to have nested if with a variable.

- **Audio -** Any number of sounds is fine; the **MINIMUM** is **ONE**. Note that sound is important - if no one can hear it, no one will use your application.
- **Data representation** – Use appropriate blocks to enable the operation of variables or lists..
- **Operators** - Your application must consist of operators.
- **Events** – Use more than TWO (2) types of "event-handler".
- **Broadcast** - Your application must consist of at least **ONE** broadcast message.
- **Make a Block** - Your application must consist of at least **ONE** custom block with parameters and should be called accordingly.

Additional Technical Requirements:

- **Input Data** – Your application should be able to allow interaction with users, even if the project overall is not interactive.

Bug free.

- Users must be able to operate all parts of the application without crashing or exhibiting unworkable behaviour or other hang-ups.
- Students should test every part of their application to be sure there are no bugs and that it is fully functional.
- Things to be consider
    - Is it easy to play or use?
    - Does everything work?
    - Be alert to the problems that might be caused by unexpected user behaviour.
    - **Validation:** When you ask the user a "Yes" or "No" question (y/n?) what will happen if they press 9?
    - **User friendly:** Include the instruction on how to operate your application.

The application should follow the **RULES** as below; ***(any violation will be reward as zero (0) mark for the student)***

1. No vulgarism
2. No inappropriate graphics and contents
3. Relevant to the topic you have chosen.
4. Application must be fully working, properly terminated and no errors.

PART B: DOCUMENTATION (15%)

The code must be documented appropriately. Prepare relevant documentation based on your application. Your documentation should consist of the following requirements:

- Assignment Cover Page
- Table of Contents
- Contents
    - **Introduction (including the title)**
    - **Stage 1 – Analysis**
        - Objective(s) of the application
        - Target users
        - Hardware, software, skills required.
        - Gantt chart for application development
    - **Stage 2: Design**
        a) Describe each sprite's functionality used in your application
        b) Design a pseudocode or flowchart
        c) Storyboard of the application
    - **Stage 3: Development**
        a) Programing Concepts with source code for explanation
    - **Stage 4: Implementation**
        a) Application user manual
        b) Screenshots of sample input/output and explanation
        c) Testing results
    - **Conclusion - Application capabilities and limitations.**

- **References** (**MINIMUM OF 3** references need to be included by using **APA-Referencing Style**)
    - All information, figures and diagrams obtained from external sources must be referenced.
    - **Reminder:** Plagiarism is a serious offense, and your assignment will automatically be awarded **zero (0)** marks.

Document formatting:

Spacing: 1.5spacing

Cover page: Font Size: up to 16pts; bold

Heading or title: Font size: up to 14pts; bold and underline

Flowchart / pseudocode: Font-style: century gothic; font-size: 10pts

Documentation Content: Font style: Time News Roman; Font size: 12pts;

Alignment: Justify

2.0 SUBMISSION REQUIREMENT

- Student is compulsory to submit the documentation and the application in SNAP! Via Moodle.
- Last submission due date: **24th November 2023 (Friday), 6:00 p.m.**
- **Late submission** will result in **marks deduction** for the individual project marks, without warning.

3.0 DELIVERABLES:

Component	Who to submit?	How to submit?	File Name
Group Project	Group Leader	1. DOC/DOCX file via Moodle. 2. Application file (link or xml) via Moodle	*IVIP_UCDF2308ICT_GroupName*

4.0 Marking Criteria

Group Project (60%)		
Component	**Breakdown**	**Description**
PART B: APPLICATION (45%)		
Code Readability		The source code is well explained through the instruction provided in the project
Creativity		Creative and has a unique design.
Storyline		The project has a clearly developed plot with an obvious beginning, middle and end.
Use of Sprites		Multiple sprites with costume change and appropriate naming for all sprites in the project. Relevant background design.
Motion of Sprites (Animation)		Animation(motion) is used effectively.
Use of Media (Sound)		Uses more than two types of media components.
User Interactivity		Uses more than two types of event handlers (Keyboard interactive) or excellent user prompts and the program responds to them well.
Use of Procedures - custom block		Custom block defined with parameter(s) and called.
Use of Procedures - broadcast message		Broadcast messages were used effectively (including the use of run / launch).
Conditional		Uses more than one “if then else” involving variables and expressions in the condition and it is working effectively.
Loops		Uses complex loops or “for each” loops with list items.
Use of Data (Variable or List)		Variables were created with meaningful names, initialized correctly, and used appropriately. List were defined and used appropriately.
Runtimes		Executes without errors.
Program termination		The program is able to terminate according to the program instruction.
Part C: Documentation (15%)		
Content	Objective	This includes accuracy and relevancy of facts and information with good use of language and grammar. Clear objective, with details on the purpose of the program and who will be the target user
	Target user	
	Hardware, software, skills	
	Gantt chart	
	Design	Very clear, understandable, readable, and organized. Pseudocode covers more than 80% of system requirements with correct logic/solution. A

		pseudocode can be easily converted into a program. Flowchart covers more than 80% of system requirements with correct logic/solution and shapes. The flowchart can be easily understanding the flow of the applications.
	Development	Program clarity, coding style, variable naming, use of modular programming technique, ability to implement required criteria and mapped according to produced design.
	Implementation	Completed all the testing. Correctness of program (i.e. ability to produce intended output).
	Conclusion	Excellent conclusions
	References	Professional formatting with the use of APA referencing style.